

ZAK MCP Gateway: Implementation Roadmap

System Architecture, Execution Lifecycle & Phase-Wise Implementation Plan

1. Executive Overview

The **ZAK MCP Gateway (TO-MCP)** is a transparent middleware proxy positioned between AI agents (Cursor, Claude Desktop, LangChain) and downstream Model Context Protocol servers. It resolves the two critical protocol flaws: **Schema Inflation** and **Payload Flooding**.

✗ Standard MCP (Without Gateway)

- **Fixed cold-start:** ~28,500 tokens of unminified Draft-07 JSON schemas.
- **Unused tools:** 91% of loaded tools are never called during a session.
- **Payload flood:** 15KB–60KB raw REST API payloads injected into memory.

✓ ZAK MCP Gateway (Optimized)

- **Fixed cold-start:** ~320 tokens (exposes only 2 universal meta-tools).
- **On-demand discovery:** Compact TypeScript signatures loaded JIT.
- **Egress projection:** JSONPath masks strip nulls and unwanted fields.

2. Three-Step Execution Lifecycle

Step 1: Just-In-Time Discovery (`mcp_search_tools`)

Agent sends a query (e.g. "create issue"). MiniSearch BM25 engine retrieves matching tools in <1ms and returns compact TypeScript signatures (~36 tokens vs ~192 tokens JSON schema).

Step 2: Execution Dispatch & Decision Gate (`mcp_call_tool`)

Gateway evaluates Tier-1 SHA-256 exact hit cache. If idempotent query is cached, returns result instantly (0 tokens, 0 latency). Otherwise, dispatches call to child MCP process.

Step 3: Egress Distillation & Projection (`project_fields`)

Gateway applies JSONPath field whitelisting, removes recursive null/empty structures, and transforms JSON arrays into compact TSV tables before returning to the model.

3. Phase-by-Phase Implementation Roadmap

Phase	Component	Key Implementation Tasks	Token / Latency Goal
Phase 1	Core Stdio Proxy & JIT Discovery (MVP)	• Connect downstream stdio processes (<code>client-pool.ts</code>) • MiniSearch BM25 lexical index (<code>search-index.ts</code>) • Expose <code>mcp_search_tools</code> & <code>mcp_call_tool</code>	-99% Startup 320 token fixed root
Phase 2	Schema Transpiler	• JSON Schema → Compact TypeScript compiler (<code>ts-transpiler.ts</code>) • Handle primitives, enums, optional fields, union types • Token usage counter & benchmarking harness	-81% Schema 36 tokens per signature
Phase 3	Egress Distiller & Decision Gate	• JSONPath <code>project_fields</code> filter (<code>projection-filter.ts</code>) • Recursive null/empty pruner (<code>null-pruner.ts</code>) • Tier-1 SHA-256 exact match cache (<code>sha-cache.ts</code>) • Idempotency safety evaluator (<code>idempotency.ts</code>)	-94% Payload 0 tokens on cache hits
Phase 4	Host Integration & Code Sandbox	• Register in Claude Desktop and Cursor configs • Multi-turn SWE-bench / developer benchmark evaluation • Phase 2 roadmap: Ephemeral QuickJS/V8 sandbox (Code Mode)	E2E Verification Production ready

4. Benchmark Performance Target

Metric	Standard MCP	ZAK Gateway	Net Advantage
Startup Context Footprint	38,450 tokens	320 tokens	-99.1% overhead
Average Response Size	14,200 tokens	840 tokens	-94.0% payload
Total 15-Turn Trajectory	576,750 tokens	18,420 tokens	-96.8% total volume
Time-to-First-Token (TTFT)	3,840 ms	410 ms	9.3x faster
Estimated Cost (Sonnet 3.5)	\$2.88	\$0.09	32x cost reduction